

소리 내어 외우는 한국사

음성으로 답하는 한국사 암기 앱 — 직접 따라 만드는 수업 교재

AI 융합디자인 실습 · 예제 프로젝트 | 준비물: 무료 코드 에디터 + 크롬 브라우저 (유료 도구·서버·계정 불필요)

이 교재로 만드는 것. 화면에 사건 이름이 잠깐 나타났다 사라지면, 학습자가 **소리 내어** 설명하고, 브라우저가 그 말을 글자로 바꿔 **핵심어가 들어갔는지**로 정답을 판정하는 웹앱입니다. 파일 4개 (HTML·CSS·JS)만으로 동작하며, **설치나 서버 없이** 브라우저로 바로 실행됩니다.

이 수업의 범위. 실제 서비스에서는 더 정확한 음성 인식을 위해 외부 유료 STT(예: 네이버 CLOVA)와 서버(백엔드)를 연동할 수 있지만, **이 교재에서는 그 과정을 생략합니다.** 대신 크롬에 내장된 **무료 음성 인식 기능 (Web Speech API)**을 사용해 누구나 똑같이 따라 만들 수 있게 했습니다.

1. 프로젝트 소개

무엇을 만드나

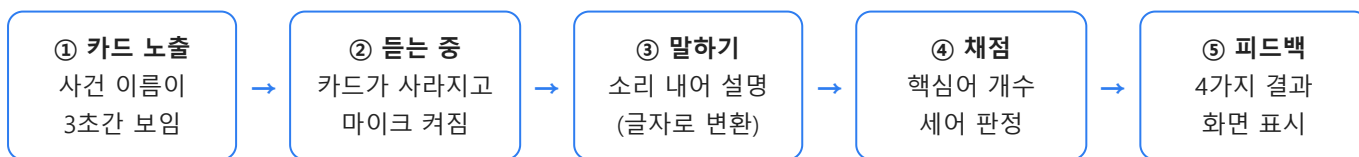
한국사 사건을 "눈으로 읽는" 대신 "입으로 설명하며" 외우는 퀴즈 앱입니다. 개항기·근대 5개 사건 (강화도 조약 · 임오군란 · 갑신정변 · 동학농민운동 · 을사늑약)을 다룹니다.

왜 이렇게 만드나 (이론적 근거)

- **발화 효과(Production Effect):** 소리 내어 말하며 외우면 눈으로만 읽을 때보다 기억이 오래 남습니다.
- **생성 효과(Generation Effect):** 직접 답을 "만들어" 꺼낼 때가, 정답을 그냥 읽을 때보다 학습에 효과적입니다.

그래서 이 앱은 "정답 보기"가 아니라 "직접 말하기"를 시킵니다.

한 문항이 진행되는 순서



5문항을 마치면 **결과 요약** 화면이 나옵니다. (완성본에는 해설·원문 팝업, 학습 통계 화면도 있습니다.)

2. 준비물

필요한 것	설명
코드 에디터	Visual Studio Code (무료). code.visualstudio.com 에서 설치. (메모장으로도 가능하지만 VS Code 권장)
웹 브라우저	Chrome (무료). 음성 인식(Web Speech API)이 크롬·엣지에서 잘 동작합니다.
제공 파일	수업예제_소스 폴더 — <code>index.html</code> , <code>styles.css</code> , <code>app.js</code> , <code>data.js</code>

유료 구독 도구(예: Claude Code)나 서버, 로그인 계정은 **필요하지 않습니다**.

3. 전체 구조 한눈에

앱은 **파일 4개**로 이루어집니다. 각자 역할이 분명히 나뉩니다.

파일	역할	비유
<code>index.html</code>	화면의 뼈대(어떤 요소가 있는지)	건물의 골조
<code>styles.css</code>	디자인(색·크기·배치)	인테리어·페인트
<code>data.js</code>	문항 내용(사건·정답 키워드·해설)	교과서 내용
<code>app.js</code>	동작(타이머·음성 인식·채점)	두뇌·전기 배선

브라우저가 `index.html` 을 열면, 그 안에서 `styles.css` (디자인)와 `data.js` · `app.js` (내용·동작)를 함께 불러옵니다.

4. 단계별로 만들기

STEP 1 폴더와 파일 준비

1. 바탕화면에 폴더를 하나 만듭니다. 예: `korean-history-app`
2. 제공된 **수업예제_소스** 폴더 안의 4개 파일을 그 폴더에 복사합니다.
3. VS Code를 열고 **파일 → 폴더 열기**로 방금 만든 폴더를 엽니다.
4. 왼쪽에 `index.html · styles.css · app.js · data.js` 4개가 보이면 준비 끝입니다.

이 교재는 각 파일이 **어떻게·왜** 그렇게 만들어졌는지 설명합니다. 전체 코드는 제공 파일에 들어 있으니, 각 단계에서 해당 파일을 열어 함께 보세요. 아래에는 **핵심 부분**을 발췌해 설명합니다.

STEP 2 문항 내용 넣기 — `data.js`

가장 먼저 "무엇을 물어볼지"를 정합니다. `data.js`에는 문항 5개가 들어 있고, 문항 하나는 이렇게 생겼습니다.

```
// data.js - 문항 하나의 구조 (강화도 조약 예시)
{
  id: "1",
  사건명: "강화도 조약",
  연도: "1876",
  정답_키워드: ["1876", "조약", "일본인", "허용", "강화도"],
  통과_기준: 3, // 5개 중 3개 이상 맞으면 "통과"
  해설: "강화도 조약은 1876년 운요호 사건을 빌미로 ...",
  한줄정리: "일본이 운요호 사건을 구실로 맺은 최초의 근대적·불평등 조약",
  보충: "운요호 사건: 일본 군함이 강화도를 침입한 사건",
  원문: [ "제1관 조선국은 자주의 나라로서 ...", "..." ],
  원문_출처: "출처: 제71회 한국사능력검정시험 기출 자료"
}
```

핵심은 딱 두 가지입니다.

- **정답_키워드**: 이 사건을 설명할 때 반드시 들어가야 할 단어들. 채점의 기준이 됩니다.
- **통과_기준**: 그 키워드 중 **몇 개 이상** 말하면 통과로 볼지. 여기서는 3으로 두어, "사건 이름만" 말해서는 통과되지 않고 연도·내용까지 설명해야 통과됩니다.

키워드는 어떻게 골랐나? 손으로 임의로 고르면 주관적이라, 사건 원문을 **형태소 분석**(문장에서 명사만 추출) → **TextRank**(중요한 단어에 점수 매기는 알고리즘, 구글 페이지랭크와 같은 원리)로 **자동 추출**했습니다. 여기에 연도·핵심 인물을 포함하는 규칙을 더했습니다. 이 수업에서는 그 **결과(정답_키워드)**만 그대로 사용하면 됩니다. (추출은 선택 심화 주제)

STEP 3 화면 뼈대 만들기 — `index.html`

앱은 여러 "화면"이 있지만 실제로는 한 페이지 안에 모두 들어있고, **한 번에 하나씩만 보이게** 전환합니다. 각 화면은 `<section class="screen">` 로 감싸고, 현재 화면에만 `active` 를 붙입니다.

```

<!-- 화면들은 모두 .screen, 현재 화면만 .active 로 보인다 -->
<section id="screen-listen" class="screen">
  <!-- 진행 표시(개항기 · 2/5) -->
  <div class="listen-status">듣는 중</div>
  <div class="listen-stage">
    <div class="wave" id="wave"></div> <!-- 소리 물결 애니메이션 -->
    <button class="mic-btn" id="mic-btn"> ...마이크 아이콘... </button>
  </div>
  <p class="listen-foot">말을 마치고 2초 멈추면 자동으로 채점돼요</p>
  <div class="recog" id="recog"> ...실시간 인식 텍스트... </div>
</section>

```

맨 아래에서 CSS와 JS 파일을 불러오는 한 줄이 중요합니다. **순서대로** 넣습니다.

```

<!-- 디자인 -->
<link rel="stylesheet" href="styles.css">
<!-- 내용(data) 먼저, 그다음 동작(app) -->
<script src="data.js"></script>
<script src="app.js"></script>

```

STEP 4 디자인 입히기 — styles.css

디자인 코드는 길기 때문에 **제공된 styles.css** 를 그대로 사용합니다. 대신 어떤 원칙으로 만들어졌는지만 이해하면 됩니다.

- **모바일 화면 틀:** 폭 390 · 높이 844(휴대폰 크기)로 가운데 배치.
- **색 규칙:** 파랑(기본), 노랑(타이머), 그리고 결과별 색 — **완전정답/통과=초록, 부분정답=주황, 모름=회색**.
- **화면 전환:** `.screen` 은 기본 숨김, `.screen.active` 만 보이게 하는 것이 핵심입니다.

```

/* styles.css 의 핵심 규칙 - 화면 하나만 보이게 */
.screen { display: none; }
.screen.active { display: flex; }

```

STEP 5 동작 로직 만들기 — app.js (가장 중요)

app.js 는 앱의 "두뇌"입니다. 네 가지 일을 합니다: **화면 전환** · **타이머** · **음성 인식** · **채점**.

5-1. 화면 전환과 진행 흐름

화면을 바꾸는 것은 `active` 클래스를 옮기는 함수 하나로 끝납니다.

```
function showScreen(name) {
  document.querySelectorAll(".screen").forEach(s => s.classList.remove("active"));
  document.getElementById("screen-" + name).classList.add("active");
}
```

진행은 **카드(3초) → 듣기 → 채점 → 피드백 → 다음 문항** 순서로, 타이머(`setTimeout`)로 자동으로 넘어갑니다.

5-2. 음성 인식 (브라우저 무료 기능)

크롬에 내장된 **Web Speech API**를 씁니다. 서버도, 키도, 설치도 필요 없습니다.

```
// 브라우저가 제공하는 음성 인식 객체
const SR = window.SpeechRecognition || window.webkitSpeechRecognition;

function startSTT() {
  recognition = new SR();
  recognition.lang = "ko-KR";           // 한국어
  recognition.interimResults = true;    // 말하는 중간 결과도 표시
  recognition.onresult = onSpeechResult; // 인식될 때마다 화면에 표시
  recognition.onend = () => { if (finalTranscript) grade(finalTranscript); };
  recognition.start();                  // 듣기 시작
}
```

마이크를 못 쓰는 환경(권한 거부·미지원 브라우저)에서는 자동으로 **키보드 입력창**이 떴서, 답을 타이핑해도 똑같이 채점됩니다. (안전장치)

5-3. 채점 — 이 앱의 심장 (AI가 아니라 규칙)

정답/오답을 AI가 판단하지 않습니다. 인식된 문장에 정답 키워드가 몇 개 들어있는지 세는 단순한 규칙입니다.

```
function grade(rawText) {
  const q = QUESTIONS[currentIndex];
  const normText = normalize(rawText); // 공백 제거 등 정리

  // 정답 키워드 중, 답변에 들어있는 것만 골라 개수를 센다
  const matched = q.정답_키워드.filter(kw => normText.includes(normalize(kw)));
  const count = matched.length;
  const total = q.정답_키워드.length; // 5
  const pass = q.통과_기준;           // 3

  let key;
  if (count === total) key = "complete"; // 5개 다 → 완전정답
  else if (count >= pass) key = "pass"; // 3~4개 → 통과
  else if (count > 0) key = "partial"; // 1~2개 → 부분정답
}
```

```
else                key = "unknown";    // 0개 → 모름

renderFeedback(key, q, matched);    // 결과 화면 표시
}
```

이 네 갈래가 그대로 **4가지 피드백 화면**(초록/초록/주황/회색)으로 이어집니다.

STEP 6 실행하기

1. 가장 간단: `index.html` 을 **더블클릭** → 크롬에서 열립니다.
2. 시작하면 **마이크 권한 허용**을 눌러 주세요. (한 번만 허용하면 됩니다)
3. 카드가 사라지면 **소리 내어** 답하고, 2초 멈추면 자동 채점됩니다.
4. 마이크가 안 되면 듣기 화면의 " **키보드로 답하기**"로 타이핑해도 됩니다.

마이크가 파일 열기(file://)에서 막힐 때가 있습니다. 그럴 땐 키보드 입력으로 진행하거나, 크롬 주소창에 `localhost` 로 여는 방식을 쓰면 됩니다. (수업에서는 키보드 입력으로도 충분합니다.)

5. 핵심 개념 정리

개념	설명
채점 = 규칙	정답 키워드가 답변에 들어있는지 개수만 센다. AI(LLM)로 "맞다/틀리다"를 판단하지 않는다.
음성 인식 = 무료 기능	크롬 내장 Web Speech API. "음성 → 글자" 변환만 담당. 서버·키 불필요.
키워드 = 알고리즘 선정	형태소 분석 + TextRank로 객관적으로 추출(+연도·인물 포함 규칙). 결과만 <code>data.js</code> 에 사용.
화면 = 하나만 보이기	모든 화면이 한 페이지에 있고 <code>.active</code> 로 하나씩 전환.

수업 범위 밖(참고). 완성작에서는 더 정확한 음성 인식을 위해 네이버 CLOVA STT를 서버(Supabase)를 거쳐 연동하고, 웹에 배포(Vercel)하기도 했습니다. 하지만 그 과정은 유료 서비스·계정·서버 설정이 필요해 **이 교재에서는 다루지 않습니다.** 궁금하면 별도 자료를 참고하세요.

6. 직접 바꿔보기 (과제·확장)

- **문항 바꾸기:** `data.js`에서 사건·정답 키워드·해설을 다른 시대로 교체해 보세요.
- **난이도 조절:** `통과_기준`을 2로 낮추면 쉬워지고, 4로 올리면 어려워집니다.
- **시간 조절:** `app.js` 맨 위 `CONFIG`의 카드 노출 시간·침묵 시간을 바꿔 보세요.
- **색·문구 바꾸기:** `styles.css`의 색 값과 화면 문구를 취향대로 수정.

7. 자주 막히는 점 (Q&A)

증상	해결
화면이 하얗게 비어요	4개 파일이 같은 폴더 에 있는지, 파일명이 정확한지(<code>app.js</code> 등) 확인.
마이크가 안 돼요	크롬/엣지 사용, 권한 허용 . 그래도 안 되면 키보드로 답하기 사용.
음성이 이상하게 인식돼요	또박또박, 조용한 곳에서. 인식이 완벽하지 않아도 핵심어만 들어가면 통과됩니다.
파이어폭스에서 마이크가 안 돼요	파이어폭스는 음성 인식 미지원 → 자동으로 키보드 입력 모드로 전환됩니다.

정리. ① `data.js`에 문항과 **정답 키워드**를 넣고, ② `index.html`로 화면 뼈대를, ③ `styles.css`로 디자인을, ④ `app.js`로 **타이머·음성 인식·키워드 채점**을 연결하면 "소리 내어 외우는 한국사"가 완성됩니다. 핵심은 **"채점은 AI가 아니라 키워드 규칙"**이라는 점입니다.

